

User Admin — Full-Stack Dev Assessment

A small admin application that lets administrators **view a list of users and manage their profiles and addresses** (one user → many addresses).

- **Backend:** Java 17 + Spring Boot 3 (in-memory store, no DB required)
 - **Frontend:** React 18 + TypeScript + Vite + MUI v6 + React Router v6 + TanStack Query + react-hook-form + Zod
-

Prerequisites

Tool	Version
Java JDK	17 or newer (tested on 26)
Maven	Not required — the included <code>./mvnw</code> wrapper downloads Maven 3.9.9 on first run
Node.js	18+ (tested on 22)
npm	9+

Note on JAVA_HOME: the wrapper auto-locates a JDK 17+ on Windows. If yours is on PATH and JAVA_HOME already points to a JDK 17+ install, that wins.

Run the backend

From Git Bash / WSL / macOS / Linux:

```
cd backend
./mvnw spring-boot:run
```

From Windows cmd / PowerShell:

```
cd backend
mvnw.cmd spring-boot:run
```

The API will be available at `http://localhost:8080`. On startup, an in-memory store is seeded with 5 users and 1–3 addresses each.

Run tests:

```
./mvnw test
```

(If you already have Maven installed, `mvn spring-boot:run` works just the same.)

Run the frontend

```
cd frontend
npm install
```

```
npm run dev
```

Open `http://localhost:5173`. Vite proxies `/api/*` to the backend on `:8080`, so you don't need to configure anything else for local development.

Type-check:

```
npm run lint
```

Production build:

```
npm run build
```

API contract

All routes are JSON, served under `/api`.

Method	Path	Body	Response
GET	<code>/users</code>	—	UserSummary[] (id, email, firstName, lastName, addressCount)
GET	<code>/users/{id}</code>	—	User (full, with addresses)
POST	<code>/users</code>	UserRequest	201 User
PUT	<code>/users/{id}</code>	UserRequest	User
DELETE	<code>/users/{id}</code>	—	204
GET	<code>/users/{id}/addresses</code>	—	Address[]
POST	<code>/users/{id}/addresses</code>	AddressRequest	201 Address
PUT	<code>/users/{id}/addresses/{addressId}</code>	AddressRequest	Address
DELETE	<code>/users/{id}/addresses/{addressId}</code>	—	204

Validation failures return 400 with a `details` map keyed by field. Missing entities return 404.

UserRequest — { email, firstName, lastName } (email validated)
AddressRequest — { label, line1, line2?, city, state?, postalCode, country }

Design notes — the User → Address flow

The modification flow is the heart of the assessment. The design choice:

One detail screen per user; addresses edited via dialogs in place.

1. **Users list** (/users) is a clean MUI `DataGrid` — name, email, and an address-count chip. Each row is clickable.
2. **Detail page** (/users/:id) loads the user and their addresses in parallel via two `TanStack Query` calls (and `Suspense`-friendly skeletons while loading).
3. **Profile form** sits at the top of the detail page. It's a `react-hook-form` form with `Zod` validation that mirrors the BE DTOs. Save / Reset only enable when the form is dirty.
4. **Addresses** render below as a responsive grid of cards. Each card shows the label, lines, city/state/postal, and country, plus inline Edit / Delete actions.
5. **Add / Edit address** opens an MUI `Dialog` rather than navigating to a new route. This keeps the admin focused on one user, avoids losing context, and matches how real admin tools work: pick a user, then perform all their edits without bouncing between routes.
6. **Delete** routes through a small `ConfirmDialog` so accidents are caught. `Snackbars` confirm success.

Why this shape

- **Pragmatic 1:N**: addresses are owned by users on the backend (`ConcurrentHashMap<UUID, User>` where each `User` holds its own `List<Address>`). The API mirrors that ownership: addresses live under `/users/{id}/addresses/*`. No orphans possible.
- **Two-tier read model**: `UserSummary` for the list (lean, fast) and `UserDto` for the detail (full, with addresses). Keeps the table cheap while the detail view has everything it needs in a single fetch — and the addresses sub-resource enables granular cache invalidation.
- **Cache invalidation**: after any address mutation, the addresses query and the parent user queries (detail + list) are invalidated. This is what keeps the address-count chip on the list page in sync.
- **Forms isolated, server-state shared**: form state lives in `react-hook-form`; server state lives in `TanStack Query`. No `Redux` or `Context` — they would be ceremony for this scope.

Stack rationale (one-liner each)

- **MUI v6** — broad component coverage out of the box; lets the work focus on layout and flow, not pixels.
- **React Router v6** — standard, declarative routing.
- **TanStack Query** — handles loading/error/cache/refetch states elegantly; turns the FE into a thin orchestration layer.
- **react-hook-form + Zod** — minimal re-renders, schema-driven validation, single source of

truth for form shape.

- **Vite** — fast dev server with a built-in `/api` proxy.

Project structure

```
full-stack-dev-assessment/
├── README.md
├── backend/
│   ├── pom.xml
│   └── src/
│       ├── main/java/com/assessment/baseservice/
│       │   ├── BaseServiceApplication.java
│       │   ├── config/           (CorsConfig, DataSeeder)
│       │   ├── controller/      (UserController, AddressController)
│       │   ├── service/         (UserService – the in-memory store)
│       │   ├── model/           (User, Address)
│       │   └── dto/              (UserDto, UserSummary, UserRequest, AddressDto,
│       │                               AddressRequest)
│       ├── exception/           (NotFoundException, GlobalExceptionHandler)
│       ├── main/resources/application.yml
│       └── test/java/...        (UserServiceTest – round-trip smoke test)
├── frontend/
│   ├── package.json, vite.config.ts, tsconfig.json, index.html
│   └── src/
│       ├── main.tsx, App.tsx, theme.ts
│       ├── api/                 (client.ts, users.ts, addresses.ts)
│       ├── types/               (index.ts – mirrors BE DTOs)
│       ├── pages/               (UserListPage, UserDetailsPage, NotFoundPage)
│       └── components/           (AppLayout, ProfileForm, AddressCard,
│                               AddressFormDialog, ConfirmDialog)
```

What I'd add with more time

- **Persistence** — swap the `ConcurrentHashMap` for an H2 + JPA layer behind the same service interface. The DTO boundary makes that a swap, not a rewrite.
- **Optimistic UI** — TanStack Query's `onMutate` for instant feedback on edits.
- **Auth** — even a simple bearer-token guard with a fake login screen would round out the "admin" framing.
- **More tests** — `@WebMvcTest` for the controllers, React Testing Library for the dialogs and forms.
- **Pagination & search** on the users list once it grows past a screenful.
- **Dockerfile** for each side + a `docker-compose.yml` for one-command spin-up.

React-SpringBoot